

REACT TO-DO LIST APPLICATION

Components, State & Props

Assignment Report

GitHub Repository

<https://github.com/khushipg04-tech/version-control-testing-project>

A practical React project demonstrating components, state management, props, and interactive todo operations.

1. Introduction

React is a JavaScript library used for building interactive user interfaces. It uses reusable components and provides a convenient way to manage changing data. This assignment demonstrates these concepts through a simple To-Do List application that allows users to add, view, and delete tasks.

2. Objectives

Understand React components; use `useState` for state management; pass data and functions with props; handle user events; understand state changes and re-rendering; and apply basic React best practices.

3. React Components

Components are reusable parts of a React interface. A To-Do List can be divided into `App`, `TodoForm`, `TodoList`, and `TodoItem`. `App` can hold the main state, `TodoForm` can collect input, `TodoList` can display the collection, and `TodoItem` can represent an individual task.

4. State Management

State is data that can change during a component's lifetime. The `useState` hook can store the `todo` array and the current input. Adding or deleting a task creates an updated state value, causing React to update the user interface.

5. Props

Props, short for properties, allow a parent component to pass data or functions to a child component. Props are read-only in the receiving component. For example, `App` can pass a `todo` object and an `onDelete` function to `TodoItem`.

6. Project Features

The application supports adding a `todo`, listing all `todos`, deleting a selected `todo`, updating the interface when state changes, and organizing the UI with reusable components.

7. Project Structure

A simple structure can be: `src/App.jsx`, `src/components/TodoForm.jsx`, `src/components/TodoList.jsx`, `src/components/TodoItem.jsx`, and `src/main.jsx`.

8. Sample React Implementation

The following example demonstrates the main functionality in one functional component.

```
import { useState } from "react";

function App() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState("");

  function addTodo(e) {
    e.preventDefault();
    if (!text.trim()) return;
    setTodos([...todos, { id: Date.now(), text }]);
    setText("");
  }

  function deleteTodo(id) {
    setTodos(todos.filter(todo => todo.id !== id));
  }

  return (
```

```

    <div>
      <h1>My To-Do List</h1>
      <form onSubmit={addTodo}>
        <input value={text}
          onChange={e => setText(e.target.value)}
          placeholder="Enter a task" />
        <button type="submit">Add</button>
      </form>
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            {todo.text}
            <button onClick={() => deleteTodo(todo.id)}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;

```

9. How the Application Works

The text state stores the input value. The todos state stores the task list. onChange updates the input state. Submitting the form calls addTodo and adds a task. map() displays tasks. The Delete button calls deleteTodo, and filter() creates a new array without the selected task. setTodos updates state and React updates the UI.

10. Props-Based Design

For a larger project, App can pass todos to TodoList and a delete function to TodoList. TodoList can pass each todo and the delete handler to TodoItem. Example: and .

11. Best Practices for Props

Treat props as read-only; use clear prop names; pass only required data; pass functions when a child needs to request a parent action; keep components focused; and use stable keys when rendering lists.

12. Testing the Application

Test adding a valid task, submitting an empty task, adding multiple tasks, deleting a task, typing in the input, and confirming that the displayed list matches state. The simple in-memory version starts empty after a page refresh because persistence is not implemented.

13. Possible Improvements

Add completed-task status, edit functionality, localStorage or database persistence, filters, a backend API, authentication, and automated component tests.

14. Learning Outcomes

The learner understands functional components, useState, state changes, props, event handling, list rendering with map(), deletion with filter(), and basic React development practices.

15. Conclusion

The React To-Do List application demonstrates core React concepts in a practical way. Components organize the interface, state stores changing todo data, and props support communication between parent and child components. The project can be extended with persistence, APIs, authentication, and automated testing.

16. Submission Details

Title: React To-Do List Application – Components, State & Props

Link: <https://github.com/khushipg04-tech/version-control-testing-project>

PDF: React_Todo_List_Application_Assignment.pdf